

# CONTROLLERS REST COMO ADAPTADORES DEL DOMINIO

## OBJETIVO

Implementar **Controllers REST** en Spring Boot que actúen como **adaptadores delgados** entre la API y el dominio. Al finalizar, los estudiantes serán capaces de:

1. Comprender el patrón **Adapter/Port** en arquitectura hexagonal
2. Implementar Controllers que **delegan toda la lógica al dominio**
3. Usar **Mappers** para convertir entre DTOs y entidades de dominio
4. Aplicar validaciones con **Bean Validation**
5. Manejar excepciones del dominio y traducirlas a respuestas HTTP apropiadas

 **Principio clave:** Los Controllers NO contienen lógica de negocio, solo adaptan HTTP ↔ Dominio.

## CONCEPTOS BÁSICOS

### Adapter Pattern (Patrón Adaptador)

Patrón que permite que interfaces incompatibles trabajen juntas. En nuestro caso: HTTP (REST) ↔ Dominio (Java).

# Controllers Delgados (Thin Controllers)

Controllers que solo se encargan de:

- Recibir la petición HTTP
- Validar formato de entrada (Bean Validation)
- Mapear DTOs → Entidades de dominio
- Delegar a la capa de aplicación
- Mapear Entidades → DTOs de respuesta
- Retornar respuesta HTTP

## Lombok

Biblioteca que reduce el código repetitivo (boilerplate) en Java mediante anotaciones procesadas en tiempo de compilación. En esta guía usaremos `@RequiredArgsConstructor`, que genera automáticamente un constructor con todos los campos `final`, ideal para **inyección de dependencias por constructor** en Spring.

**Referencia:** [Project Lombok](#)

## MapStruct

Framework de mapeo entre objetos Java que genera código en tiempo de compilación, evitando reflexión en runtime.

**Referencia:** [MapStruct Documentation](#)

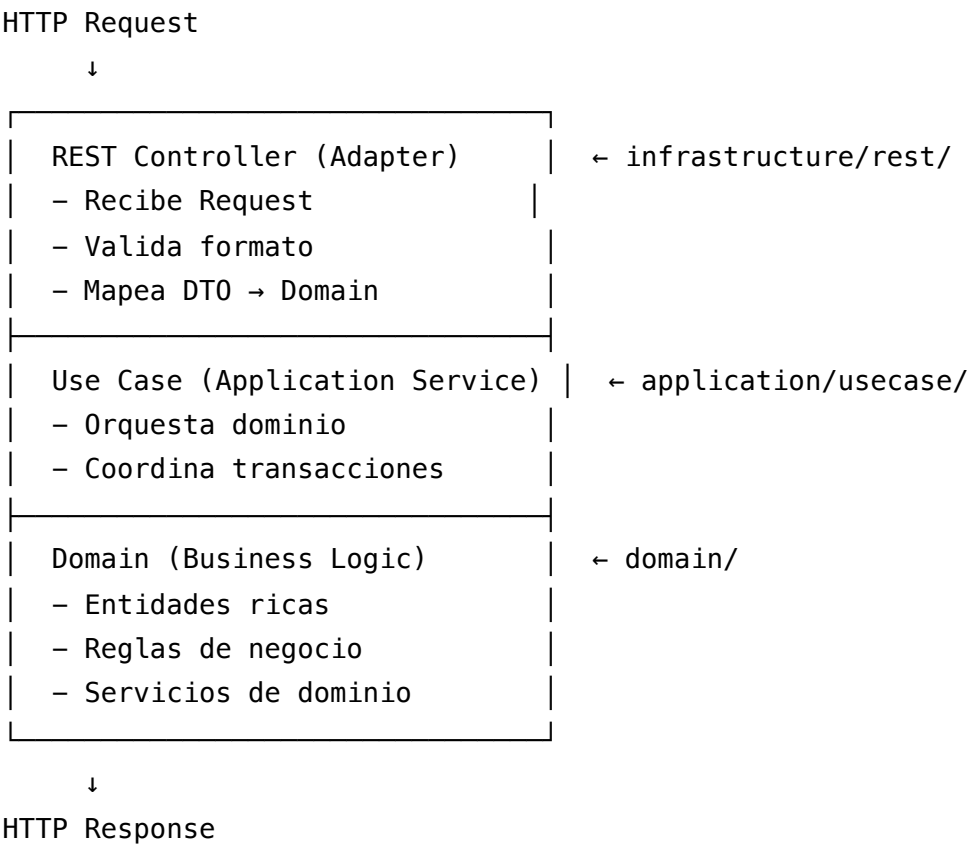
## Bean Validation

Estándar de Jakarta EE para validar objetos mediante anotaciones declarativas.

**Referencia:** [Jakarta Bean Validation](#)

# CONTEXTUALIZACIÓN TEÓRICA

## Arquitectura: Infrastructure → Application → Domain



# Controller Delgado vs. Controller Grueso

## ❌ Controller Grueso (Anti-patrón)

```
@RestController
public class SolicitudController {
    private final SolicitudRepository repo;

    @PostMapping("/solicitudes")
    public ResponseEntity<?> crear(@RequestBody SolicitudDTO dto) {
        // ❌ Lógica de negocio en el controller
        if (dto.getDescripcion().length() < 20) {
            return ResponseEntity.badRequest().body("Descripción muy corta");
        }

        // ❌ Construcción manual de entidad
        Solicitud s = new Solicitud();
        s.setDescripcion(dto.getDescripcion());
        s.setEstado("REGISTRADA");
        s.setCodigo("SOL-" + System.currentTimeMillis());

        // ❌ Acceso directo a repositorio desde controller
        repo.save(s);

        return ResponseEntity.ok(s);
    }
}
```

## ✅ Controller Delgado (DDD)

```
@RestController
@RequestMapping("/api/solicitudes")
@RequiredArgsConstructor // Lombok genera el constructor automáticamente
public class SolicitudController {
    private final CrearSolicitudUseCase crearSolicitudUseCase;
    private final SolicitudMapper mapper;

    @PostMapping
    public ResponseEntity<SolicitudDetalleResponse> crear(
        @Valid @RequestBody CrearSolicitudRequest request) {

        // 1. Delegar al Use Case (capa de aplicación)
        Solicitud solicitud = crearSolicitudUseCase.ejecutar(request);

        // 2. Mapear dominio → DTO respuesta
        SolicitudDetalleResponse response = mapper.toDetalleResponse(solicitud);

        // 3. Retornar con código HTTP apropiado
        URI location = ServletUriComponentsBuilder
            .fromCurrentRequest()
            .path("/{id}")
            .buildAndExpand(solicitud.getId())
            .toUri();

        return ResponseEntity.created(location).body(response);
    }
}
```

# PARTE 1: IMPLEMENTACIÓN DE CONTROLLER

## Estructura del Controller

🔪 **Nota:** Las dependencias de **Lombok** y **MapStruct** ya fueron configuradas en el proyecto desde la **Guía 03**. Verifique que su archivo `build.gradle` incluye `compileOnly 'org.projectlombok:lombok'` y `annotationProcessor 'org.projectlombok:lombok'`.

```

package com.uniquindio.triage.infrastructure.rest;

import com.uniquindio.triage.application.usecase.*;
import com.uniquindio.triage.application.dto.request.*;
import com.uniquindio.triage.application.dto.response.*;
import com.uniquindio.triage.domain.model.solicitud.Solicitud;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

import jakarta.validation.Valid;
import java.net.URI;
import java.util.List;

@RestController
@RequestMapping("/api/solicitudes")
@RequiredArgsConstructor // Genera constructor con todos los campos final
public class SolicitudController {

    private final CrearSolicitudUseCase crearSolicitudUseCase;
    private final ClasificarSolicitudUseCase clasificarSolicitudUseCase;
    private final AsignarResponsableUseCase asignarResponsableUseCase;
    private final ObtenerSolicitudUseCase obtenerSolicitudUseCase;
    private final ListarSolicitudesUseCase listarSolicitudesUseCase;
    private final SolicitudMapper mapper;

    // ===== CREATE =====

    @PostMapping
    public ResponseEntity<SolicitudDetalleResponse> crearSolicitud(
        @Valid @RequestBody CrearSolicitudRequest request) {

        Solicitud solicitud = crearSolicitudUseCase.ejecutar(request);
        SolicitudDetalleResponse response = mapper.toDetalleResponse(solicitud);

        URI location = ServletUriComponentsBuilder
            .fromCurrentRequest()
            .path("/{id}")
            .buildAndExpand(solicitud.getId())
            .toUri();

        return ResponseEntity.created(location).body(response);
    }

```

```
}
```

```
// ===== READ =====
```

```
@GetMapping("/{id}")
```

```
public ResponseEntity<SolicitudDetalleResponse> obtenerSolicitudPorId(  
    @PathVariable String id) {
```

```
    Solicitud solicitud = obtenerSolicitudUseCase.ejecutar(id);  
    SolicitudDetalleResponse response = mapper.toDetalleResponse(solicitud);
```

```
    return ResponseEntity.ok(response);
```

```
}
```

```
@GetMapping
```

```
public ResponseEntity<List<SolicitudResumenResponse>> listarSolicitudes(  
    @RequestParam(required = false) EstadoSolicitud estado,  
    @RequestParam(required = false) NivelPrioridad prioridad) {
```

```
    List<Solicitud> solicitudes = listarSolicitudesUseCase.ejecutar(estado, prioridad);  
    List<SolicitudResumenResponse> response = mapper.toResumenResponseList(solicitudes);
```

```
    return ResponseEntity.ok(response);
```

```
}
```

```
// ===== OPERACIONES DE NEGOCIO =====
```

```
@PutMapping("/{id}/clasificar")
```

```
public ResponseEntity<SolicitudDetalleResponse> clasificarSolicitud(  
    @PathVariable String id,  
    @Valid @RequestBody ClasificarSolicitudRequest request) {
```

```
    Solicitud solicitud = clasificarSolicitudUseCase.ejecutar(id, request);  
    SolicitudDetalleResponse response = mapper.toDetalleResponse(solicitud);
```

```
    return ResponseEntity.ok(response);
```

```
}
```

```
@PutMapping("/{id}/asignar")
```

```
public ResponseEntity<SolicitudDetalleResponse> asignarResponsable(  
    @PathVariable String id,  
    @Valid @RequestBody AsignarResponsableRequest request) {
```

```

        Solicitud solicitud = asignarResponsableUseCase.ejecutar(id, request);
        SolicitudDetalleResponse response = mapper.toDetalleResponse(solicitud);

        return ResponseEntity.ok(response);
    }

    @GetMapping("/{id}/historial")
    public ResponseEntity<List<EventoHistorialResponse>> obtenerHistorial(
        @PathVariable String id) {

        Solicitud solicitud = obtenerSolicitudUseCase.ejecutar(id);
        List<EventoHistorialResponse> historial = mapper.toHistorialResponseList(
            solicitud.obtenerHistorial()
        );

        return ResponseEntity.ok(historial);
    }
}

```

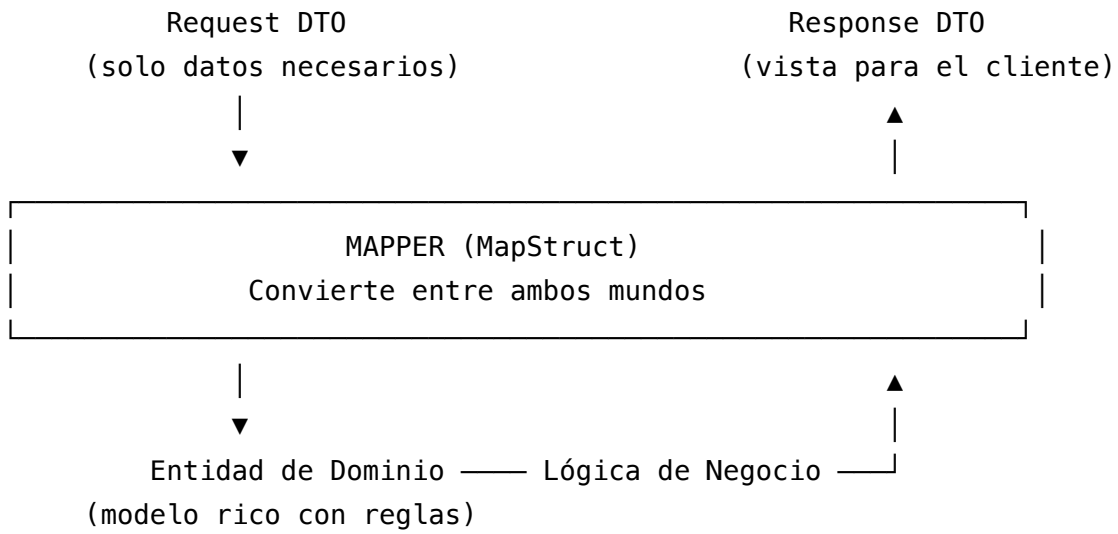
## PARTE 2: MAPPERS CON MAPSTRUCT

### ¿Por qué es necesario mapear objetos entre capas?

En una arquitectura hexagonal, las entidades de dominio representan el **modelo de negocio** con sus reglas, invariantes y comportamiento. Exponer directamente estas entidades como respuestas REST genera varios problemas:

1. **Acoplamiento API ↔ Dominio:** Cualquier cambio en la entidad de dominio (renombrar un campo, agregar una relación) modifica automáticamente la API, rompiendo contratos con los clientes.
2. **Fuga de información:** Las entidades pueden contener datos internos (auditoría, referencias a otros agregados, estados intermedios) que **no deben** ser visibles para el consumidor de la API.
3. **Representaciones diferentes:** Una misma entidad puede necesitar múltiples vistas según el contexto. Por ejemplo, un listado muestra un resumen ( `SolicitudResumenResponse` ), mientras que el detalle muestra la información completa ( `SolicitudDetalleResponse` ).
4. **Protección del dominio:** Los DTOs de entrada ( `Request` ) permiten recibir **solo los datos necesarios** para una operación, sin que el cliente pueda manipular campos que el dominio debe controlar (como el estado o la fecha de creación).





🎯 **Regla práctica:** Nunca retorne una entidad de dominio directamente desde un Controller. Siempre use DTOs de respuesta.

# Interface Mapper

```
package com.uniquindio.triage.infrastructure.rest.mapper;

import com.uniquindio.triage.application.dto.response.*;
import com.uniquindio.triage.domain.model.solicitud.*;
import org.mapstruct.*;

import java.util.List;

@Mapper(componentModel = "spring")
public interface SolicitudMapper {

    // Mapeo Dominio → Response DTO
    @Mapping(target = "tipoNombre", source = "tipo.nombre")
    @Mapping(target = "solicitanteNombre", source = "solicitante.nombreCompleto")
    @Mapping(target = "responsableNombre", source = "responsable.nombreCompleto")
    @Mapping(target = "totalEventosHistorial", expression = "java(solicitud.obtenerHist
SolicitudDetalleResponse toDetalleResponse(Solicitud solicitud);

    @Mapping(target = "tipoNombre", source = "tipo.nombre")
    @Mapping(target = "descripcionBreve", expression = "java(solicitud.getDescripcion())
    @Mapping(target = "solicitanteNombre", source = "solicitante.nombreCompleto")
    @Mapping(target = "responsableNombre", source = "responsable.nombreCompleto")
    @Mapping(target = "nivelPrioridad", source = "prioridad.nivel")
    SolicitudResumenResponse toResumenResponse(Solicitud solicitud);

    List<SolicitudResumenResponse> toResumenResponseList(List<Solicitud> solicitudes);

    // Mapeo EventoHistorial → Response
    @Mapping(target = "usuarioNombre", source = "responsable.nombreCompleto")
    EventoHistorialResponse toHistorialResponse(EventoHistorial evento);

    List<EventoHistorialResponse> toHistorialResponseList(List<EventoHistorial> eventos
}
```

## Mapper Generado Automáticamente

MapStruct genera en tiempo de compilación:

```
// Generado automáticamente por MapStruct
@Component
public class SolicitudMapperImpl implements SolicitudMapper {

    @Override
    public SolicitudDetalleResponse toDetalleResponse(Solicitud solicitud) {
        if (solicitud == null) {
            return null;
        }

        SolicitudDetalleResponse response = new SolicitudDetalleResponse();
        response.setId(solicitud.getId());
        response.setCodigo(solicitud.getCodigo().getValor());
        response.setTipoNombre(solicitud.getTipo().getNombre());
        // ... más asignaciones automáticas

        return response;
    }
}
```

## PARTE 3: VALIDACIÓN CON BEAN VALIDATION

### Anotaciones Comunes

 **Recordatorio:** Este DTO fue diseñado en la **Guía 07** (Parte 2). Aquí lo revisamos en contexto de la validación con Bean Validation.

```
package com.uniquindio.triage.application.dto.request;

import jakarta.validation.constraints.*;

public record CrearSolicitudRequest(

    @NotNull(message = "El tipo de solicitud es obligatorio")
    Long tipoSolicitudId,

    @NotBlank(message = "La descripción es obligatoria")
    @Size(min = 20, max = 1000,
        message = "La descripción debe tener entre 20 y 1000 caracteres")
    String descripcion,

    @NotNull(message = "El canal de origen es obligatorio")
    CanalOrigen canalOrigen

) {}
```

# Validaciones Personalizadas

```
// Anotación personalizada
@Target({ElementType.FIELD})
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = CodigoSolicitudValidator.class)
public @interface ValidCodigoSolicitud {
    String message() default "Código de solicitud inválido";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

// Validador
public class CodigoSolicitudValidator
    implements ConstraintValidator<ValidCodigoSolicitud, String> {

    private static final Pattern PATTERN =
        Pattern.compile("SOL-\\d{4}-\\d{5}");

    @Override
    public boolean isValid(String value, ConstraintValidatorContext context) {
        if (value == null) {
            return true; // @NotNull se encarga de null
        }
        return PATTERN.matcher(value).matches();
    }
}
```

# PARTE 4: MANEJO DE EXCEPCIONES

## Global Exception Handler

```
package com.uniquindio.triage.infrastructure.rest.exception;

import com.uniquindio.triage.domain.exception.*;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    // Manejo de errores de validación
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> handleValidationErrors(
        MethodArgumentNotValidException ex) {

        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(error ->
            errors.put(error.getField(), error.getDefaultMessage())
        );

        Map<String, Object> response = new HashMap<>();
        response.put("timestamp", LocalDateTime.now());
        response.put("status", HttpStatus.BAD_REQUEST.value());
        response.put("errors", errors);

        return ResponseEntity.badRequest().body(response);
    }

    // Manejo de excepciones del dominio
    @ExceptionHandler(SolicitudNoEncontradaException.class)
    public ResponseEntity<Map<String, Object>> handleSolicitudNoEncontrada(
        SolicitudNoEncontradaException ex) {
```

```

        return buildErrorResponse(
            HttpStatus.NOT_FOUND,
            ex.getMessage()
        );
    }

```

```

@ExceptionHandler(TransicionInvalidaException.class)
public ResponseEntity<Map<String, Object>> handleTransicionInvalida(
    TransicionInvalidaException ex) {

    return buildErrorResponse(
        HttpStatus.BAD_REQUEST,
        ex.getMessage()
    );
}

```

```

@ExceptionHandler(PermisoInsuficienteException.class)
public ResponseEntity<Map<String, Object>> handlePermisoInsuficiente(
    PermisoInsuficienteException ex) {

    return buildErrorResponse(
        HttpStatus.FORBIDDEN,
        ex.getMessage()
    );
}

```

```

// Excepciones genéricas
@ExceptionHandler(Exception.class)
public ResponseEntity<Map<String, Object>> handleGenericException(
    Exception ex) {

    return buildErrorResponse(
        HttpStatus.INTERNAL_SERVER_ERROR,
        "Error interno del servidor"
    );
}

```

```

private ResponseEntity<Map<String, Object>> buildErrorResponse(
    HttpStatus status, String message) {

    Map<String, Object> response = new HashMap<>();
    response.put("timestamp", LocalDateTime.now());
    response.put("status", status.value());
}

```

```
response.put("error", status.getReasonPhrase());  
response.put("message", message);  
  
return ResponseEntity.status(status).body(response);  
}  
}
```



# PARTE 5: TESTING DE CONTROLLERS

## Test con @WebMvcTest

```
package com.uniquindio.triage.infrastructure.rest;

import com.uniquindio.triage.application.usecase.CrearSolicitudUseCase;
import com.uniquindio.triage.infrastructure.rest.mapper.SolicitudMapper;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(SolicitudController.class)
class SolicitudControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private CrearSolicitudUseCase crearSolicitudUseCase;

    @MockBean
    private SolicitudMapper mapper;

    @Test
    void deberiaCrearSolicitudCuandoDatosValidos() throws Exception {
        // Arrange
        String requestJson = """
            {
                "tipoSolicitudId": 1,
                "descripcion": "Solicito homologación de asignatura",
                "canalOrigen": "CORREO_ELECTRONICO"
            }
            """;
    }
```

```

        Solicitud solicitudMock = crearSolicitudMock();
        when(crearSolicitudUseCase.ejecutar(any())).thenReturn(solicitudMock);

        // Act & Assert
        mockMvc.perform(post("/api/solicitudes")
            .contentType(MediaType.APPLICATION_JSON)
            .content(requestJson))
            .andExpect(status().isCreated())
            .andExpect(header().exists("Location"))
            .andExpect(jsonPath("$.codigo").exists());
    }

    @Test
    void deberiaRetornar400CuandoDescripcionMuyCorta() throws Exception {
        // Arrange
        String requestJson = """
            {
                "tipoSolicitudId": 1,
                "descripcion": "Muy corto",
                "canalOrigen": "CSU"
            }
            """;

        // Act & Assert
        mockMvc.perform(post("/api/solicitudes")
            .contentType(MediaType.APPLICATION_JSON)
            .content(requestJson))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.errors.descripcion").exists());
    }
}

```

## EJERCICIOS PRÁCTICOS

### Ejercicio 1: Implementar Endpoint de Cierre

Crear el endpoint PUT /api/solicitudes/{id}/cerrar

**Requisitos:**

- DTO de request con campo `observacion` (obligatorio, mínimo 10 caracteres)
- Delegar a `CerrarSolicitudUseCase`
- Retornar `SolicitudDetalleResponse`

## Ejercicio 2: Agregar Paginación

Modificar `listarSolicitudes` para soportar paginación:

```
@GetMapping
public ResponseEntity<Page<SolicitudResumenResponse>> listarSolicitudes(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "20") int size,
    @RequestParam(required = false) EstadoSolicitud estado) {
    // Implementar con Pageable
}
```

## PRECAUCIONES Y RECOMENDACIONES

1. **Controllers delgados:** Nunca poner lógica de negocio en controllers
2. **Validación:** Usar `@Valid` en todos los `@RequestBody`
3. **Códigos HTTP:** Usar códigos semánticos (201 Created, 404 Not Found, 400 Bad Request)
4. **Headers:** Incluir `Location` en respuestas 201
5. **Mappers:** Usar `MapStruct` para evitar mapeo manual propenso a errores

## EVALUACIÓN O RESULTADO

Al finalizar, los estudiantes deben:

1.  Tener implementado `SolicitudController` completo
2.  Usar `MapStruct` para mapeo DTO ↔ Dominio
3.  Aplicar validaciones con Bean Validation
4.  Manejar excepciones del dominio apropiadamente
5.  Escribir tests unitarios de controllers

# PRÓXIMA ACTIVIDAD

Para la **Guía 09 (Application Layer - Casos de Uso)**, preparar:

1. **Leer:** Vernon, V. (2013). *Implementing Domain-Driven Design*, Capítulo 4: "Architecture"
2. **Investigar:** Diferencia entre Application Service y Domain Service

## REFERENCIAS BIBLIOGRÁFICAS

1. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
2. Richardson, L., & Ruby, S. (2007). *RESTful Web Services*. O'Reilly Media.
3. Spring Framework Documentation: [Spring MVC](#)
4. MapStruct Documentation: <https://mapstruct.org/>
5. Jakarta Bean Validation: <https://beanvalidation.org/>